Machine learning model for detecting Sign Language Letters, based on the MNIST database

Pogoreanu Rares
56IT4T

The Data Set



Fig 1: The American Manual Alphabet

The MNIST database (Modified National Institute of Standards and Technology database) is a large database with handwritten digits, and is commonly used for training and testing machine learning models. The standard MNIST dataset contains images representing the alphabet, compressed to 28X28 pixels.

This database expanded to include other means of communication, such as sign language. Our dataset is also composed of 28X28 pixels, but instead of written letters, we have labeled pictures of hand postures. We have approx. 7171 validation pictures that we do not train with, and approximately 27456 labeled pictures for training.

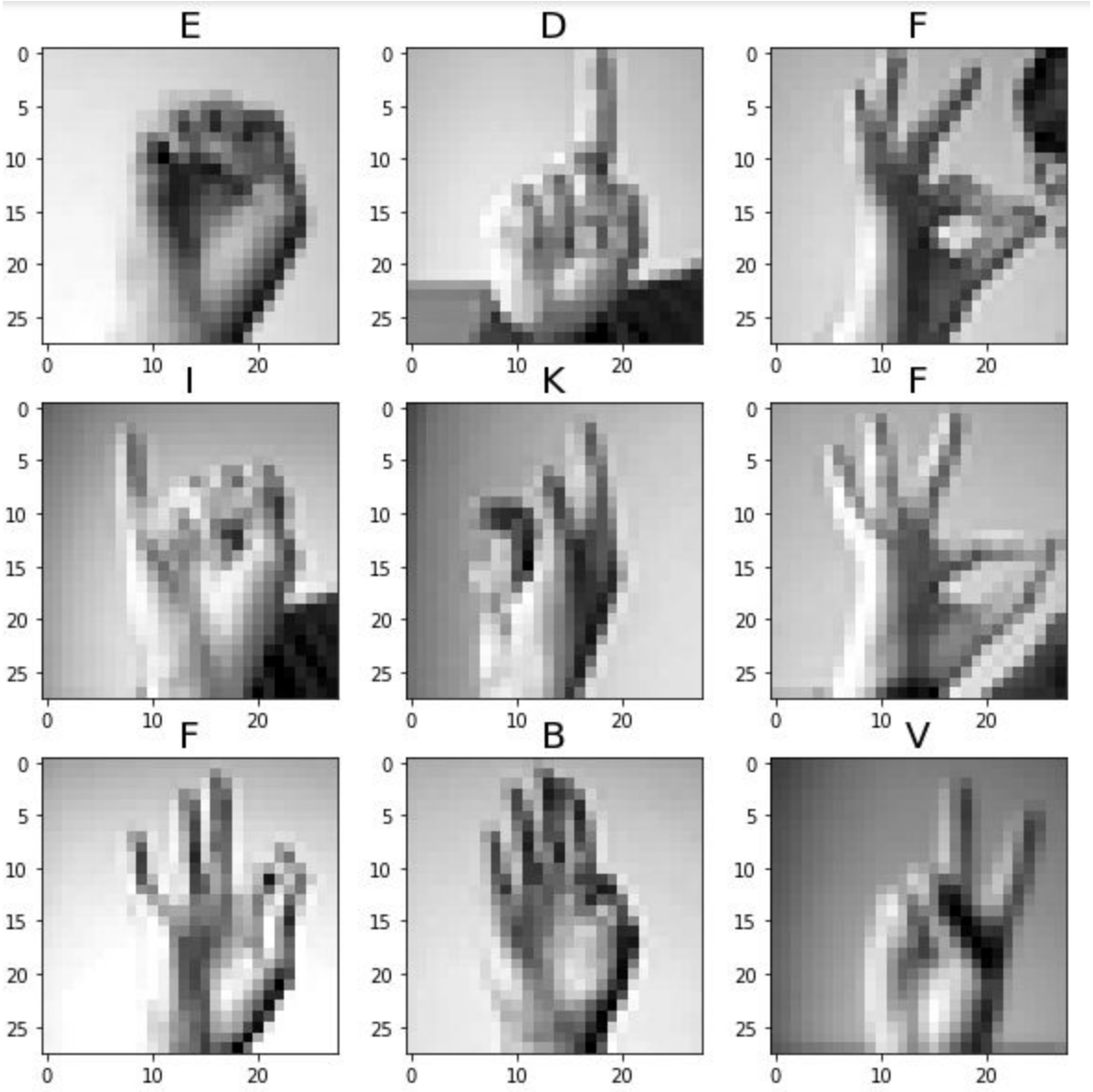


Fig2: Normalised images from the data set.

The **American Manual Alphabet** (**AMA**) contains 26 letters, but J anz Z cannot be used because they are represented with a moving gesture. The dataset format is patterned to match closely with the classic MNIST. Each training and test case represents a label (0-25) as a one-to-one map for each alphabetic letter A-Z (and no cases for 9=J or 25=Z because of gesture motions).

Tensorflow

TensorFlow is a free and open-source software library for machine learning. It can be used across a range of tasks but has a particular focus on training and inference of deep neural networks. In this project, it is used to train the model responsible for identifying and classifying the signs.

Google Colaboratory is a jupyter-notebook compatible solution, that provides free training of machine learning models for free, in the cloud. It even has dedicated GPU's and TPU's to help with the computations. This is a great solution for limited resources situations (laptop or weak computer), because it handles almost anything.

Here we have cells of python code, and we will train our solution using this method.

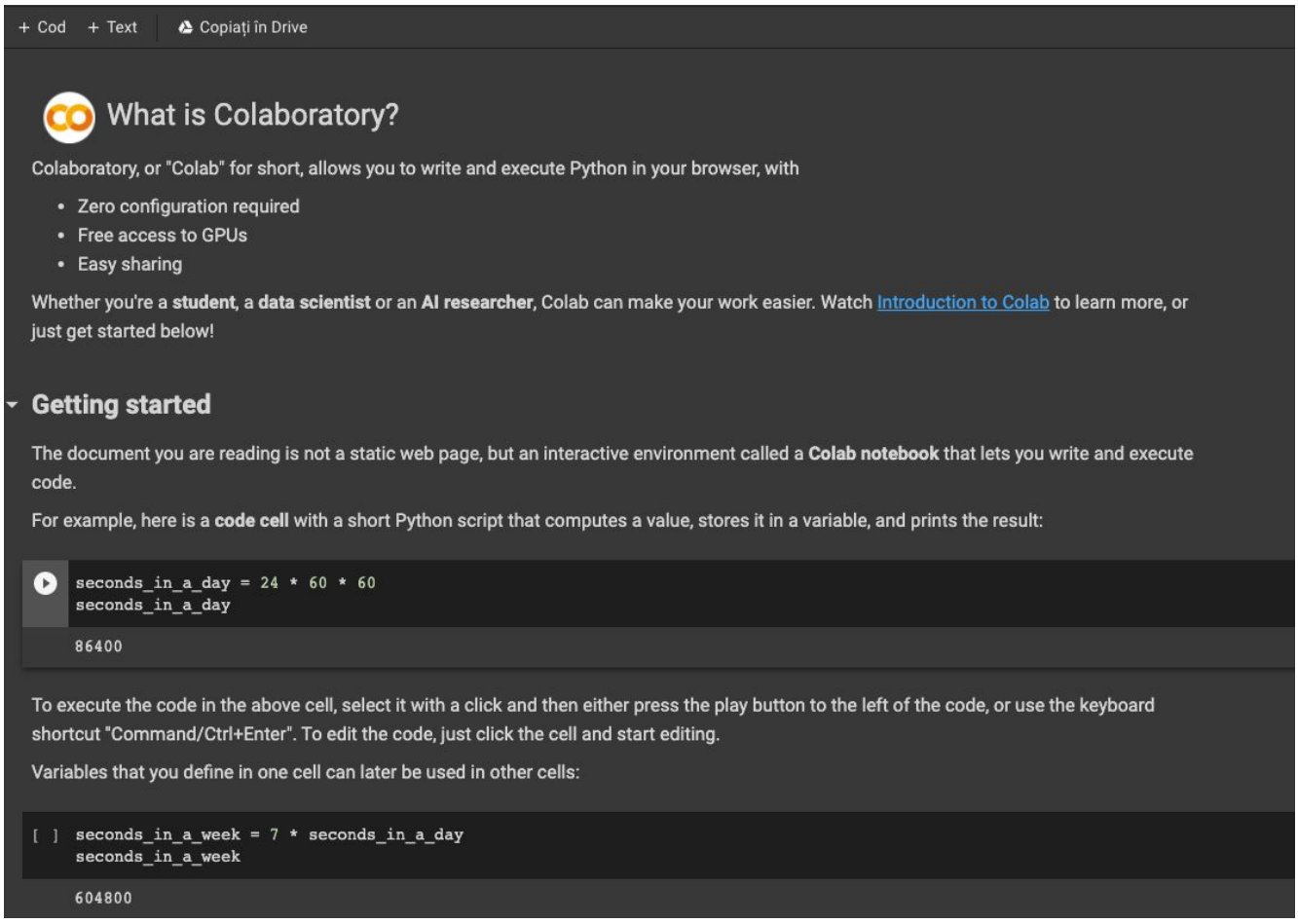


Fig 3: Google "Colab"

Architecture

For our model, a Sequential model is appropriate for a plain stack of layers where each layer has exactly one input tensor and one output tensor.

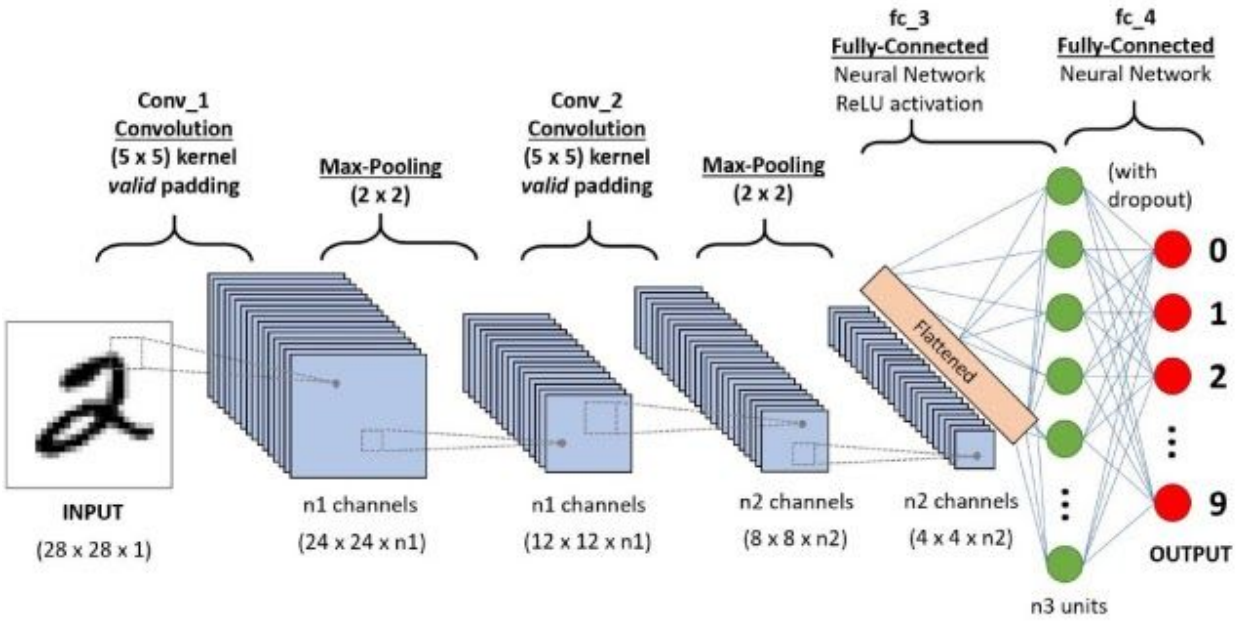


Fig 4: A standard model

```
model = Sequential()


model.add(Conv2D(filters = 256,kernel_size = (5,5),padding = 'same',activation = 'relu',
model.add(MaxPooling2D())
model.add(Conv2D(filters = 128,kernel_size = (5,5),padding = 'same',activation = 'relu')
model.add(MaxPooling2D())
model.add(Conv2D(filters = 64,kernel_size = (5,5),padding = 'same',activation = 'relu'))
model.add(MaxPooling2D())
model.add(Conv2D(filters = 32,kernel_size = (5,5),padding = 'same',activation = 'relu'))


model.add(Flatten())

model.add(Dense(units = 256,activation = 'relu'))
model.add(Dense(units = 128,activation = 'relu'))
model.add(Dropout(0.3))
model.add(Dense(24,activation = 'softmax'))

model.summary()
```

This model contains 4 internal layers, that use the Rectified Linear Unit (ReLu) activation threshold, a filter at each stage, and Max pooling in between.

Maxpooling is a technique that “pools together” the data from a few adjacent blocks and turns it into the max of that pool.

After that we flatten the array into a vector, and after that we have two more layers with relu activation, followed by the output layer that uses the SoftMax activation.

Training

```
model.compile(optimizer='adam',metrics = ['accuracy'],loss = 'categorical_crossentropy')
model.fit(train_datagen.flow(X_train,y_train,batch_size = 300),validation_data=(X_test,y_te

Epoch 1/50
24/24 [==============================] - 6s 234ms/step - loss: 1.7225 - accuracy: 0.6103 -
Epoch 2/50
24/24 [==============================] - 5s 229ms/step - loss: 0.5584 - accuracy: 0.8181 -
Epoch 3/50
24/24 [==============================] - 5s 225ms/step - loss: 0.3465 - accuracy: 0.8862 -
Epoch 4/50
11/24 [============>.................] - ETA: 1s - loss: 0.2955 - accuracy: 0.8983
```

Fig 6: Model fitting

Here we use the modified images to train the model. In each Epoch, THE entire dataset is passed through the network. More Epochs will make the model more accurate, especially if we use modified images, but there is also a risk of …

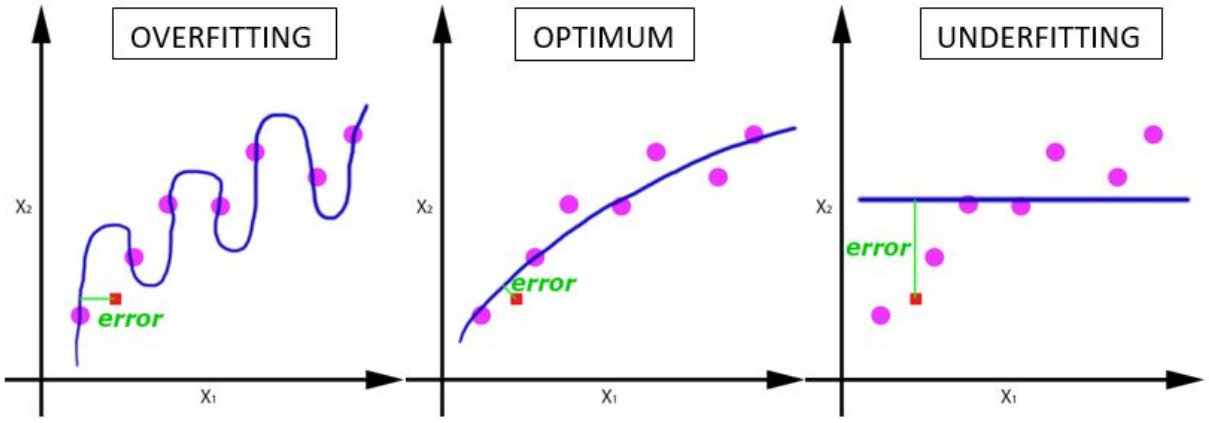


Fig 7: Overfitting, underfitting and good training session, 2-D representation

Overfitting is the usually unwanted effect of over-training and Learning the dataset way too closely, leading to poor performance in normal circumstances.

Testing

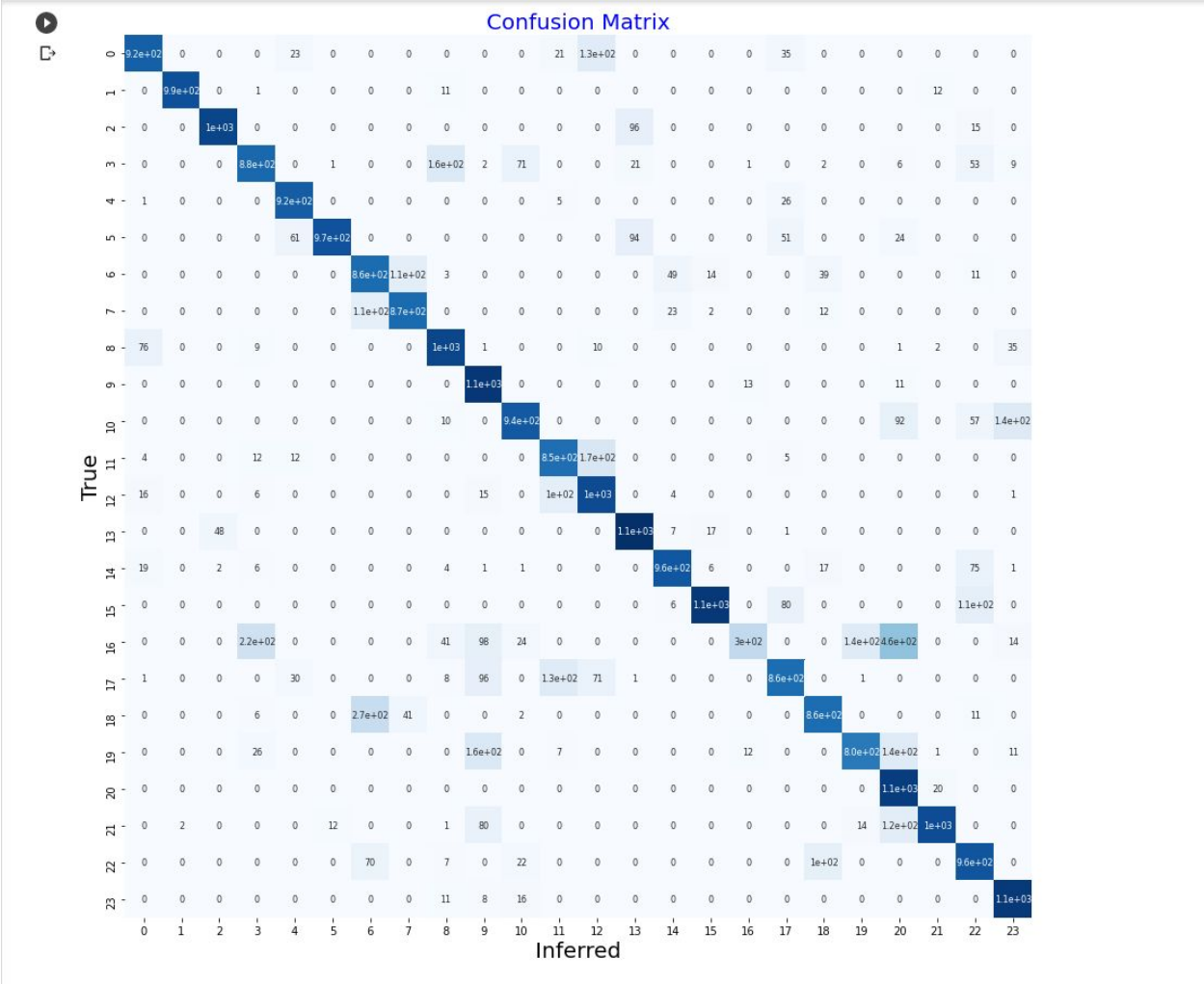


Fig 8: Confusion matrix

Results

The model recognises images from the “test” dataset, data that was not seen previously by the ML agent until now. There are a few false-positives, but that is to be expected.



Fig 9: results and tests

There is a miss on the L sign, where the model interpreted it as Y, but overall it works.

Resources:

https://www.kaggle.com/datamunge/sign-language-mnist
https://www.kaggle.com/ecco96/cnn-for-beginner-tensorflow-keras-99-accuracy/edit
https://www.youtube.com/watch?v=QfNvhPx5Px8
https://github.com/tensorflow/tensorflow/releases/tag/v2.4.1
https://heartbeat.fritz.ai/how-tensorflow-lite-optimizes-neural-networks-for-mobile-machine-learning-e6ffa7f8ee12

And of course, https://stackoverflow.com/ for random questions